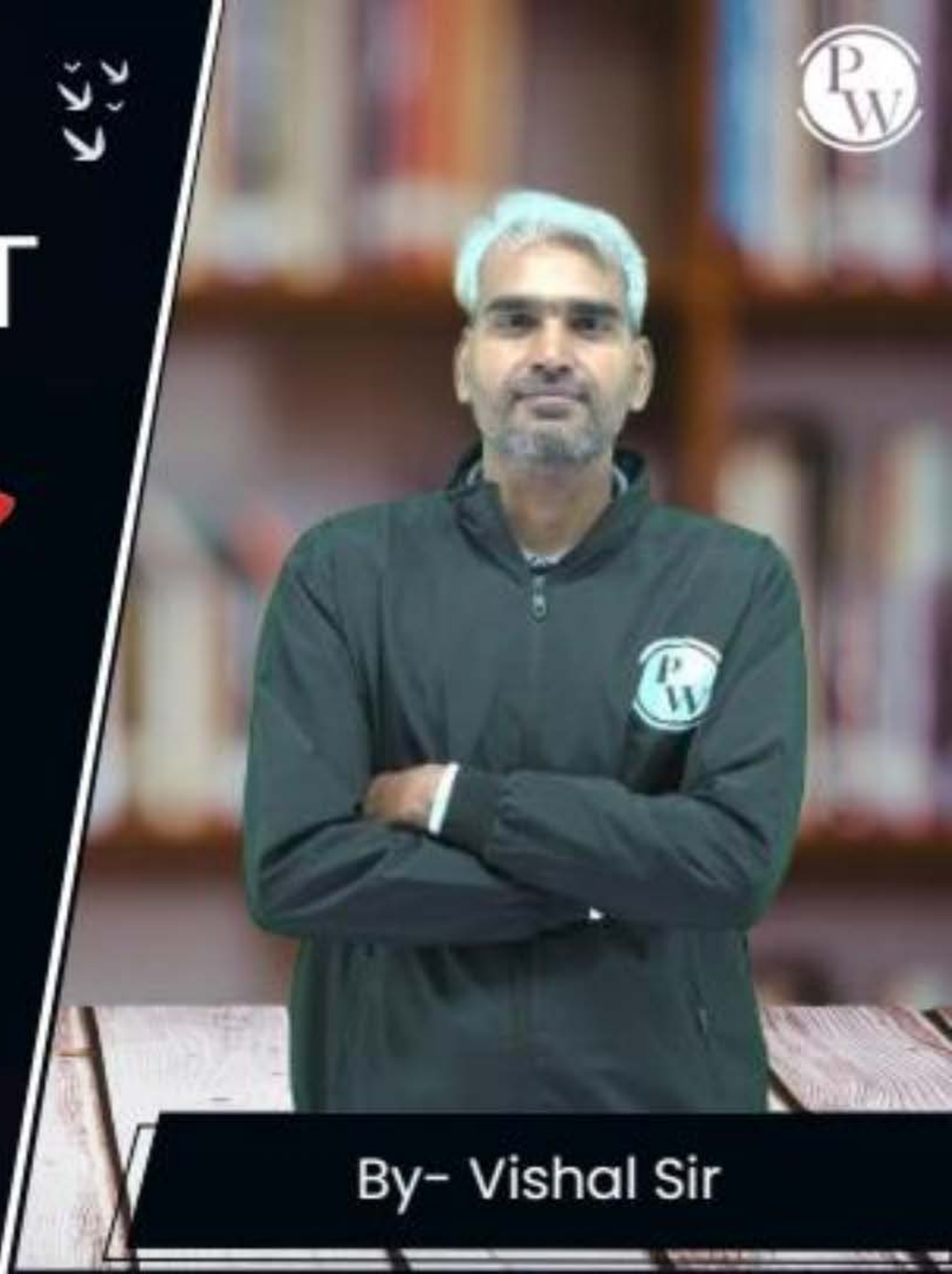
Computer Science & IT



Database Management System

B Tree and B+ Tree

Lecture No. 02



By- Vishal Sir

Recap of Previous Lecture



Topic Deletion from B tree

Topic Insertion in B+ tree

Slide

Topics to be Covered



Topic Deletion from B+ tree

Slide

Insertion in to B+ tree

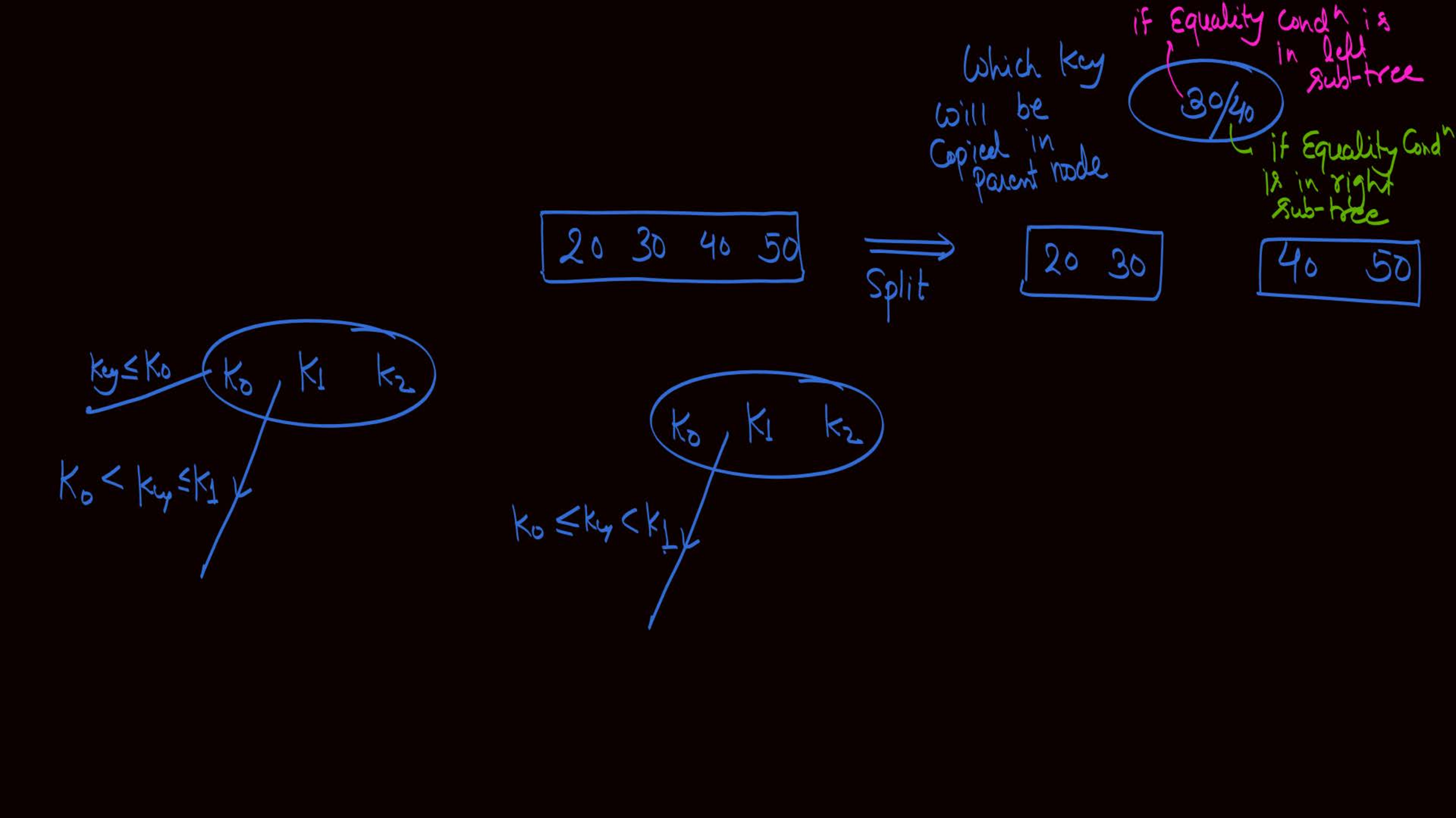


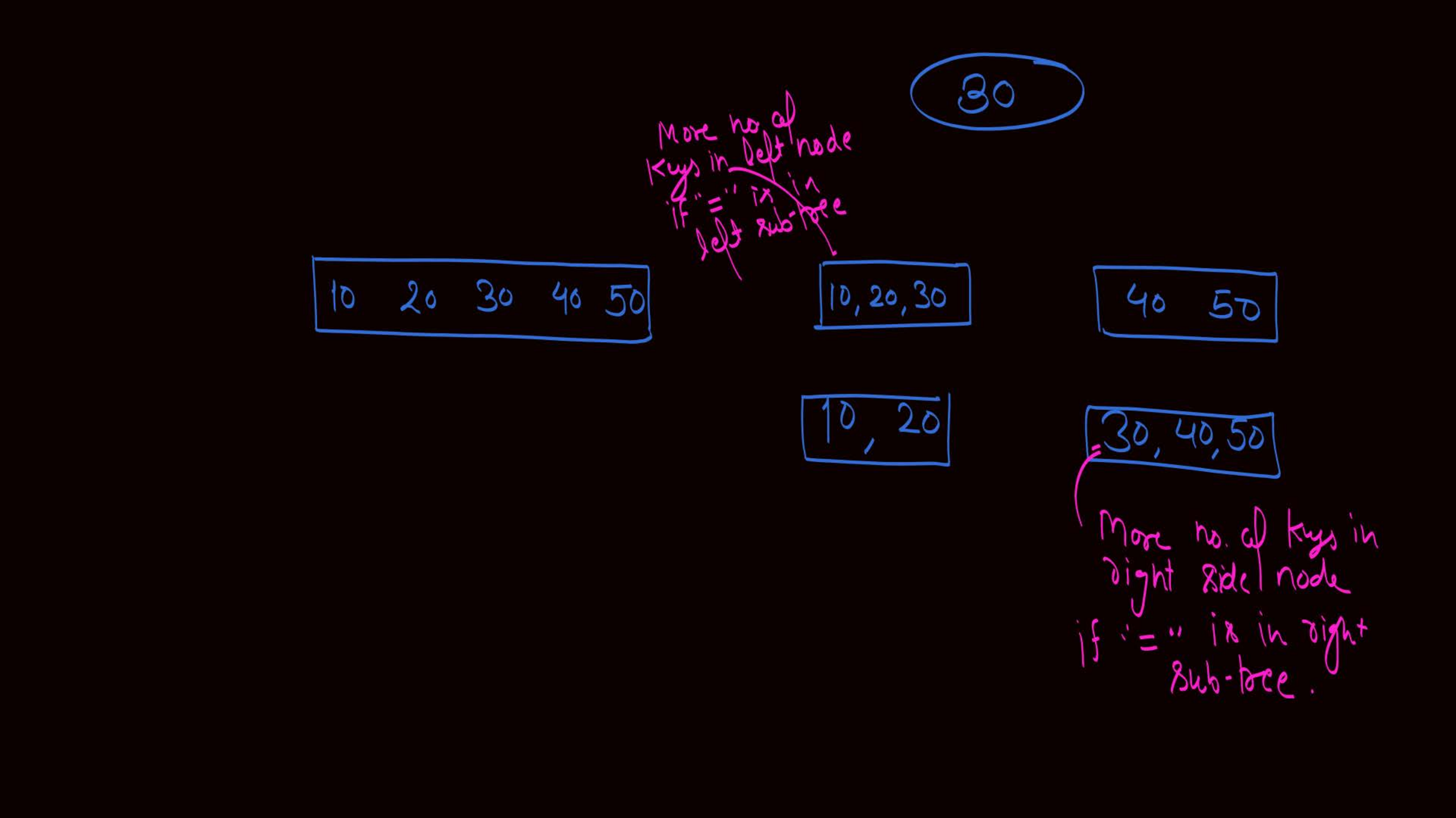
Topic : Insertion in B+ tree

- Insertion will be performed at leaf node, therefore search for appropriate leaf node
1. If leaf node is not full then insert the key into the leaf node in ascending order.
2. If leaf node is full,
    a) Insert the key in the ascending order.
    b) Split the node into two parts, and copy the median position key into the parent node, Because of this if there is overflow in the parent node then split the parent node as well.

because each key must be present in leaf node
∴ We do not promote it to parent node

Note:- While splitting the internal node, the median position key will be promoted (Not Copied) to its parent node


Which key will be Copied in parent node
if Equality Cond^n is in left sub-tree
30/40
if Equality Cond^n is in right sub-tree
20 30 40 50
Split
20 30
40 50
Key ≤ K0
K0 K1 K2
K0 < Key ≤ K1
K0 K1 K2
K0 ≤ Key < K1


30
More no of keys in left node if "=" is in left sub-tree
10 20 30 40 50
10, 20, 30
40 50
10, 20
30, 40, 50
More no. of keys in right side node if "=" is in right sub-tree.

#Q. Let,

- Order of internal node of B+ tree = 3, and
- Order of leaf node of B+ tree = 2

Insert 5, 15, 25, 35, 45 into the B+ tree in the same order

Order of internal node = 3 →

Max no. of keys in internal node = $3 - 1 = 2$

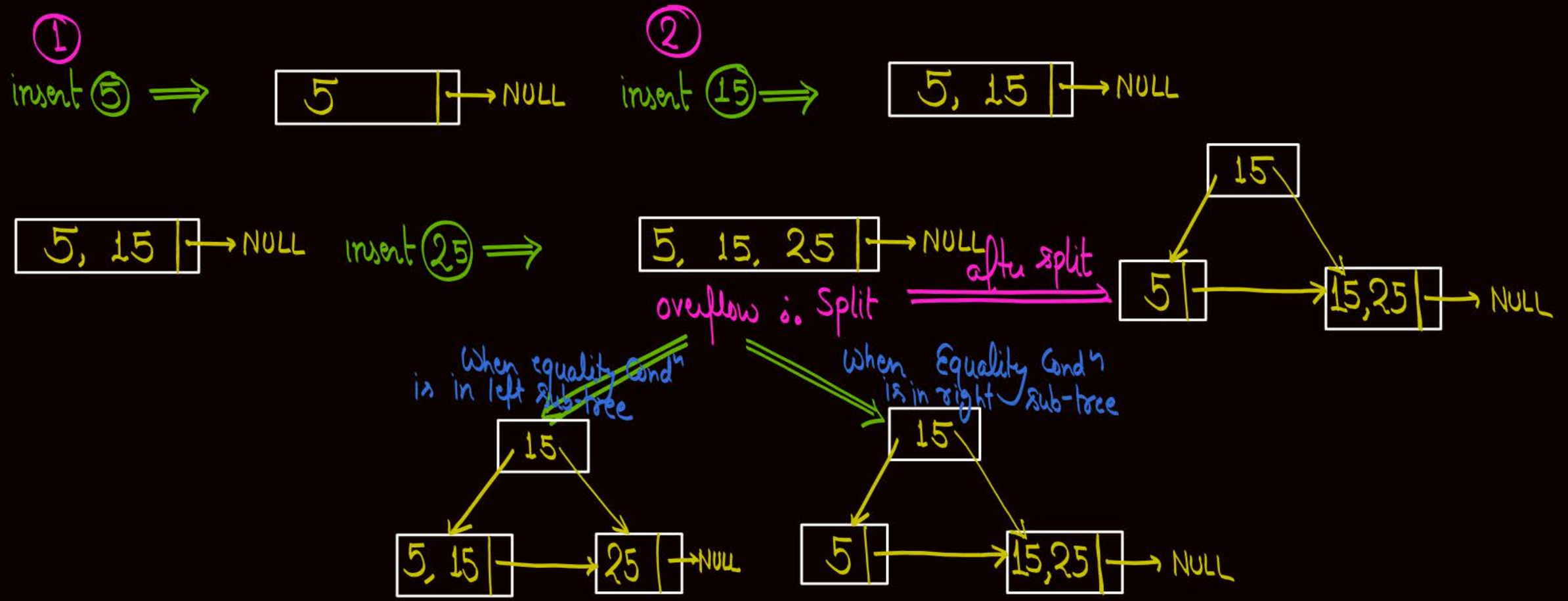
Min no. of keys required in internal node = $\lceil \frac{3}{2} \rceil - 1 = 2 - 1 = 1$

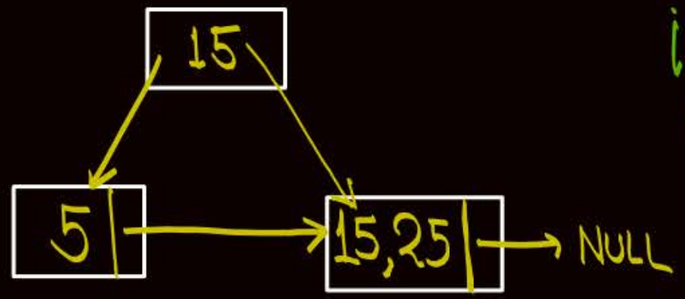
Order of leaf node = 2 →

Max no. of keys possible in leaf node = order of leaf node = 2

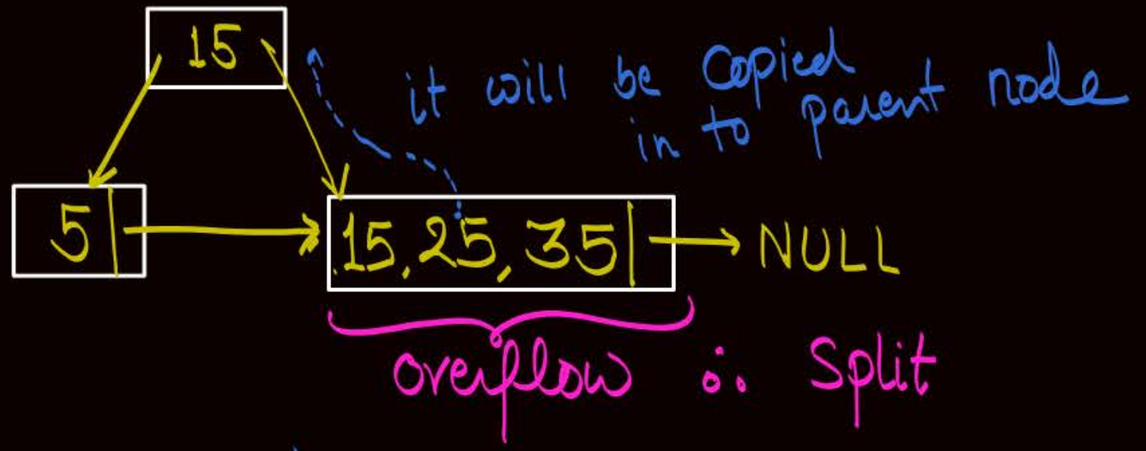
Min no. of keys required in leaf node = $\lceil \frac{\text{order of leaf node}}{2} \rceil = \lceil \frac{2}{2} \rceil = 1$

→ Min no. of keys in root node = 1

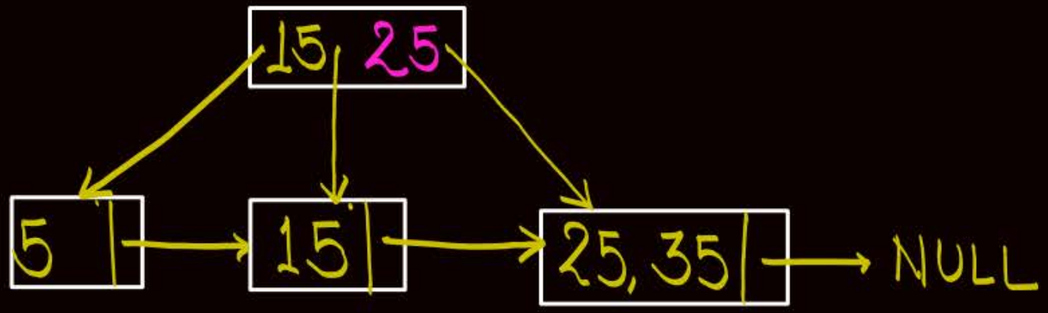

1
insert 5
5
NULL
2
insert 15
5, 15
NULL
5, 15
NULL
insert 25
5, 15, 25
NULL
overflow i. Split
after split
15
5
15,25
NULL
When equality cond^n is in left sub-tree
15
5, 15
25
NULL
When Equality Cond^n is in right sub-tree
15
5
15,25
NULL

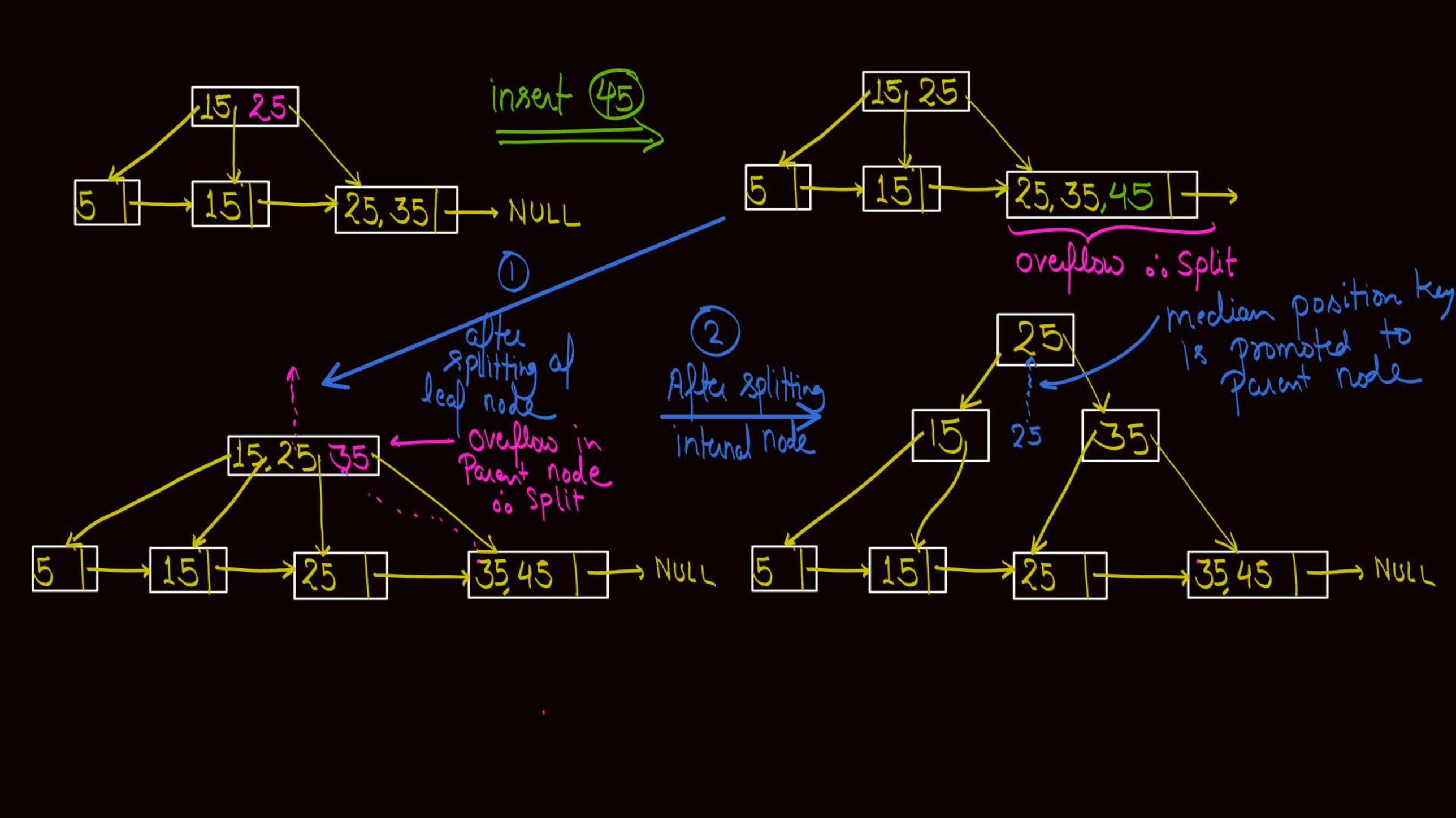


insert (35)



↓ after split




15, 25
5
15
25, 35
NULL
insert 45
15, 25
5
15
25, 35, 45
overflow ∴ Split
1
after splitting of leaf node
overflow in Parent node ∴ Split
15, 25, 35
5
15
25
35, 45
NULL
2
After splitting internal node
median position key is promoted to Parent node
25
15
25
35
5
15
25
35, 45
NULL



Topic : Deletion in B+ tree

- While deleting a key from B+ tree there are two different cases.
- Case 1 : When key to be deleted is present only at a leaf node.
- Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (Keys in the internal node are just dummy keys)



Topic : Deletion in B+ tree

- Case 1 : When key to be deleted is present only at a leaf node.
  - 1. If leaf node contains more than the minimum number of keys required, then simply delete the key.
  - 2. If leaf node contains exact minimum number of keys required, then delete the key and borrow from the immediate siblings if they can help (left then right order). (re-distribute) {When we borrow from the immediate sibling we may have to adjust the key in the parent node such that Search property is preserved}
  - 3. If siblings can not help then merge with one of its sibling. Because of this merging if parent node becomes deficient then parent node will borrow from its immediate siblings (if possible) or merge with one of its siblings, it can go up to root node.

When immidiate siblings can not help.



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

1. If leaf node contains more than the minimum number of keys required, then simply delete the key from leaf node as well as from internal node. Because of deletion from internal node an empty space will get created, fill that empty space of internal node with in-order successor (or predecessor)

When Equality cond^{n} is in right sub-tree

when Equality Cond^{n} is in left sub-tree



Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

2. If leaf node contains exact minimum number of keys required and any of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node, borrow the key from the sibling and fill that empty space of internal node with in-order successor (or predecessor)

{we may need to update key in parent node}



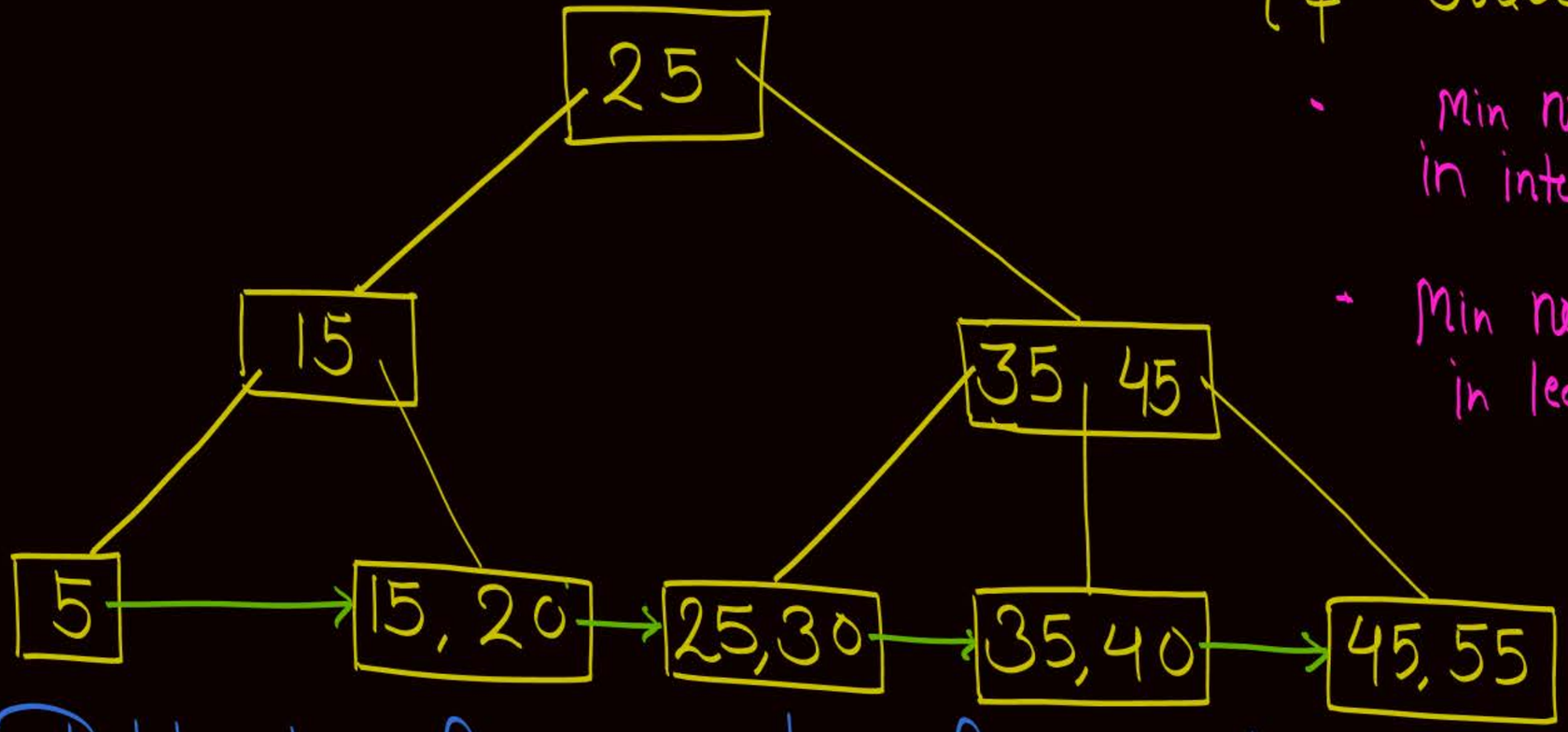
Topic : Deletion in B+ tree

Case 2 : When key to be deleted is present in a leaf node as well as in an internal node. (we need to delete the key from both places)

3. If leaf node contains exact minimum number of keys required and none of the immediate sibling can help, then simply delete the key from leaf node as well as from internal node and merge the leaf node with one of its sibling, it may create deficiency in the parent node therefore repeat the process for parent node as usual.

- If empty space created in internal node is not handled while merging the parent node then fill it with in-order successor (or predecessor).

Q:- Consider the following B+ tree { let, Order of internal node = 3 & Order of leaf node = 2



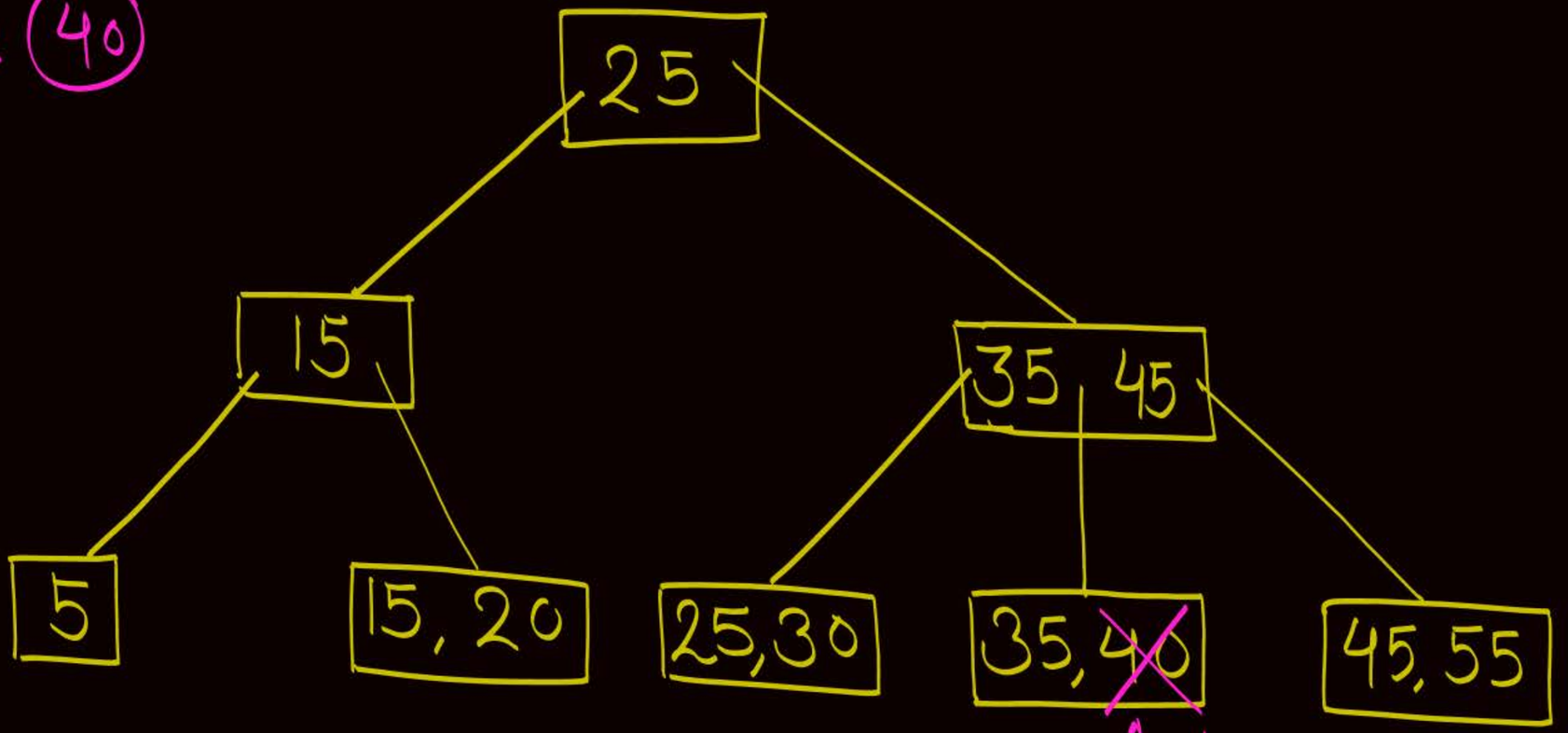
- Min no. of keys in internal = $\lceil \frac{3}{2} \rceil - 1 = 2 - 1 = 1$
- Min no. of keys in leaf node = $\lceil \frac{2}{2} \rceil = 1$

Delete the following keys from B+ tree

40, 5, 45, 35, 25, 55 in the same order

Delete: 40, 5, 45, 35, 25, 55

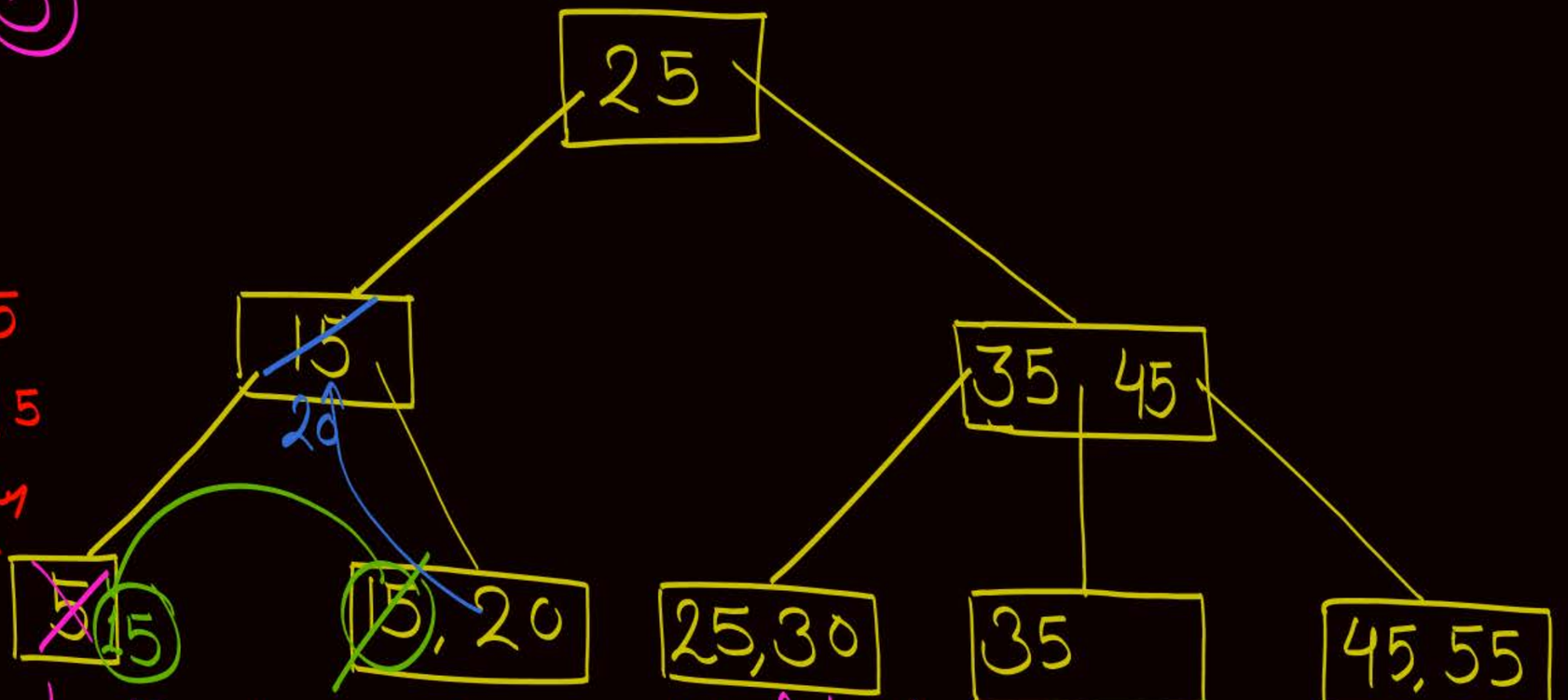
delete (40)



'40' is present only at leaf level.
& Corresponding leaf node contain more than minimum no. of keys
∴ Simply delete

Delete: ~~46~~, 5, 45, 35, 25, 55

delete (5)

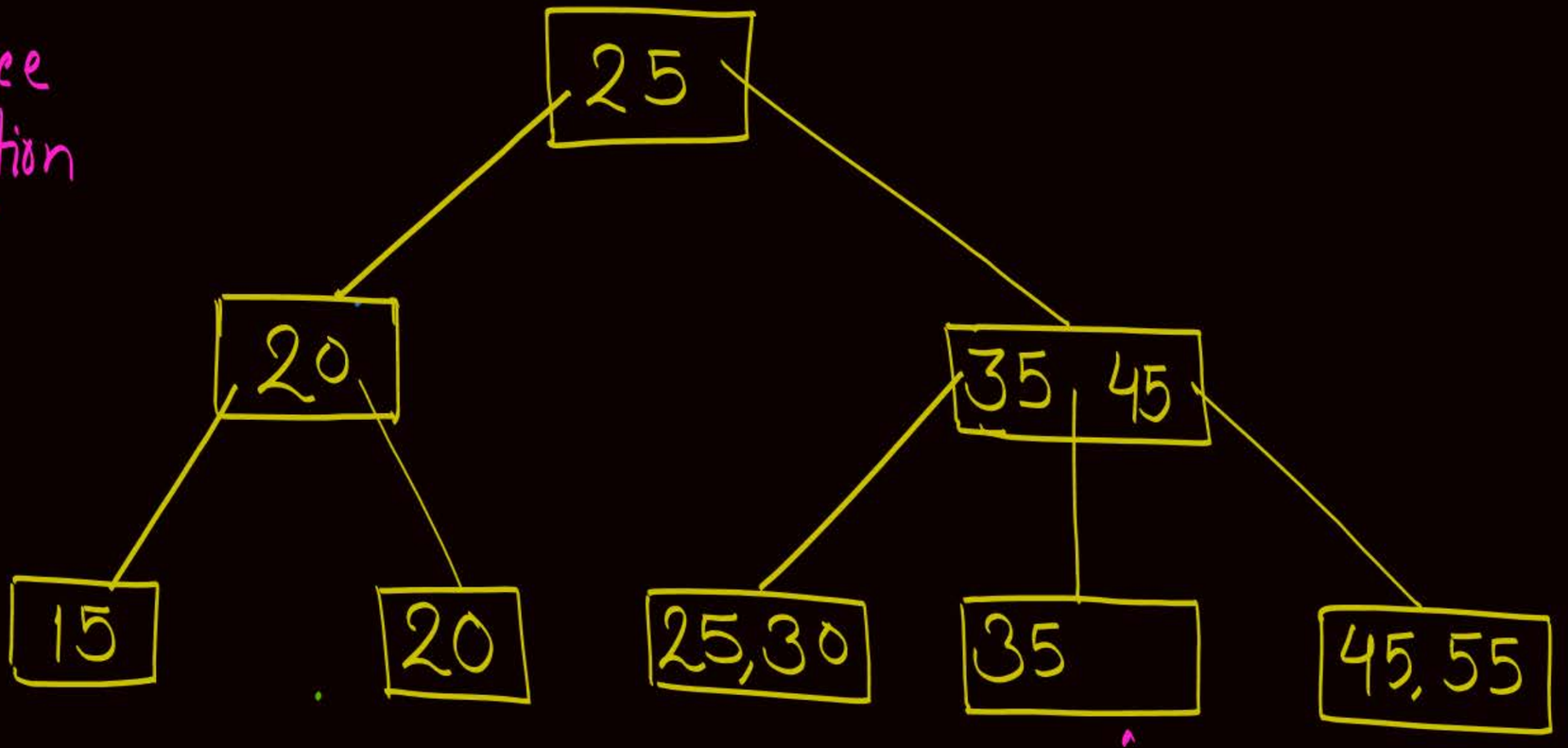


① delete 5
② Borrow 15
③ update key in parent

L 5 is present only at leaf level.
& corresponding leaf node contain Exact minimum no. of keys ∴ after deletion there will be deficiency. but right sibling can help ∴ Borrow from right sibling and update the key in parent (if required)

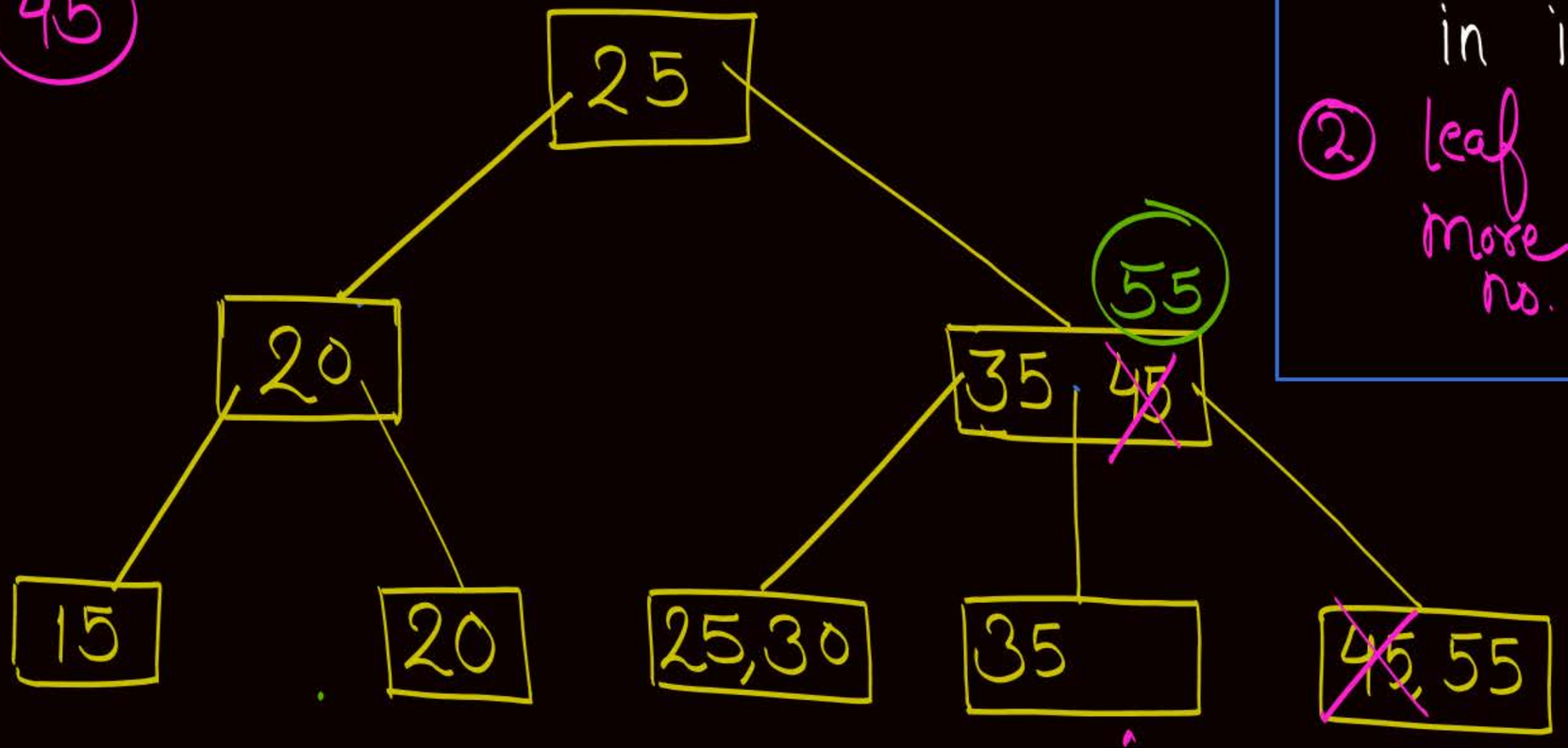
Delete: ~~40~~, 5, 45, 35, 25, 55

Final tree after deletion of "5"



Delete: ~~46~~, ~~5~~, 45, 35, 25, 55

delete (45)



① 45 is present in leaf node as well as in internal node

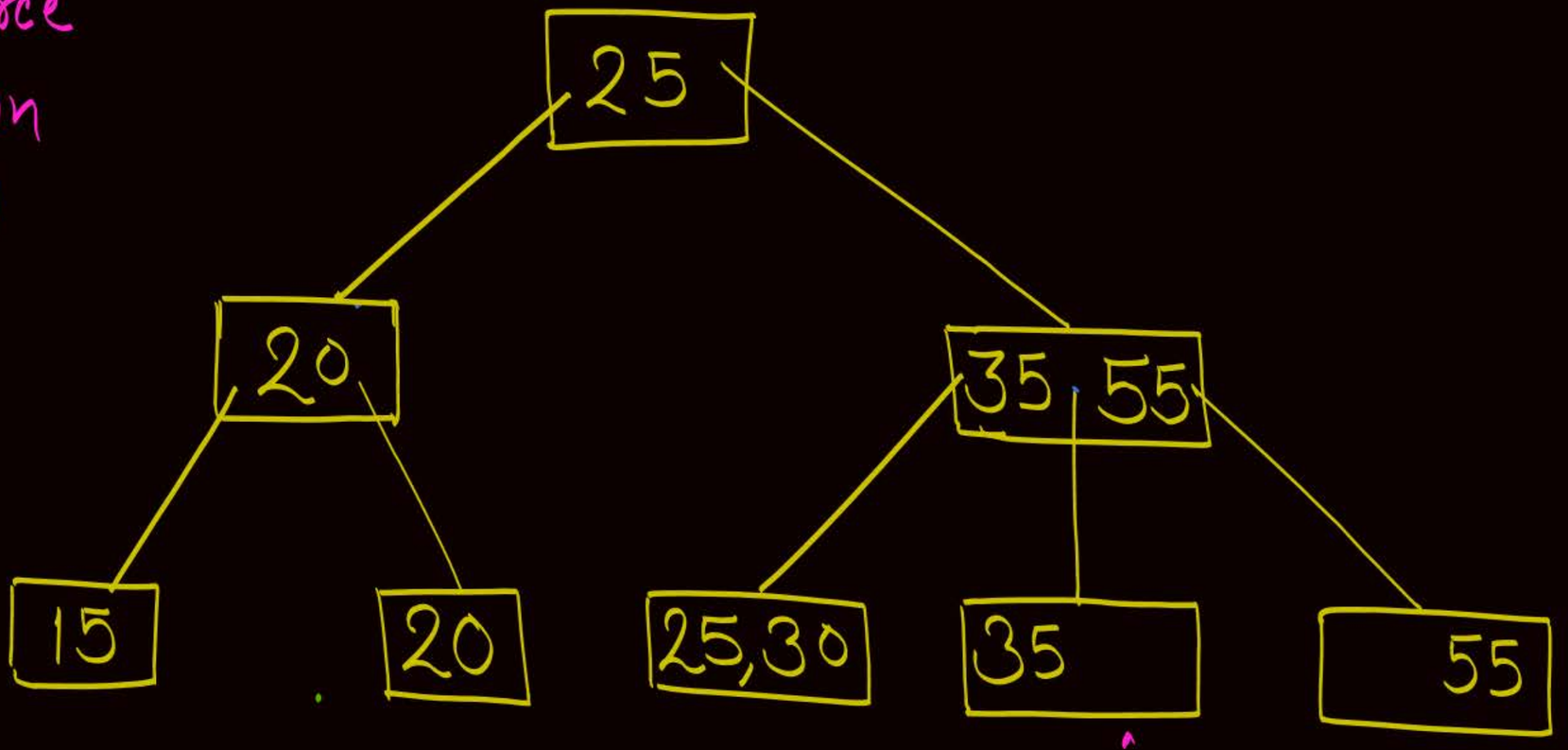
② leaf node Contains more than minimum no. of keys

∴ ① delete 45 from both places.

② Fill the Empty space in internal node using inorder successor ("=" in right subtree)

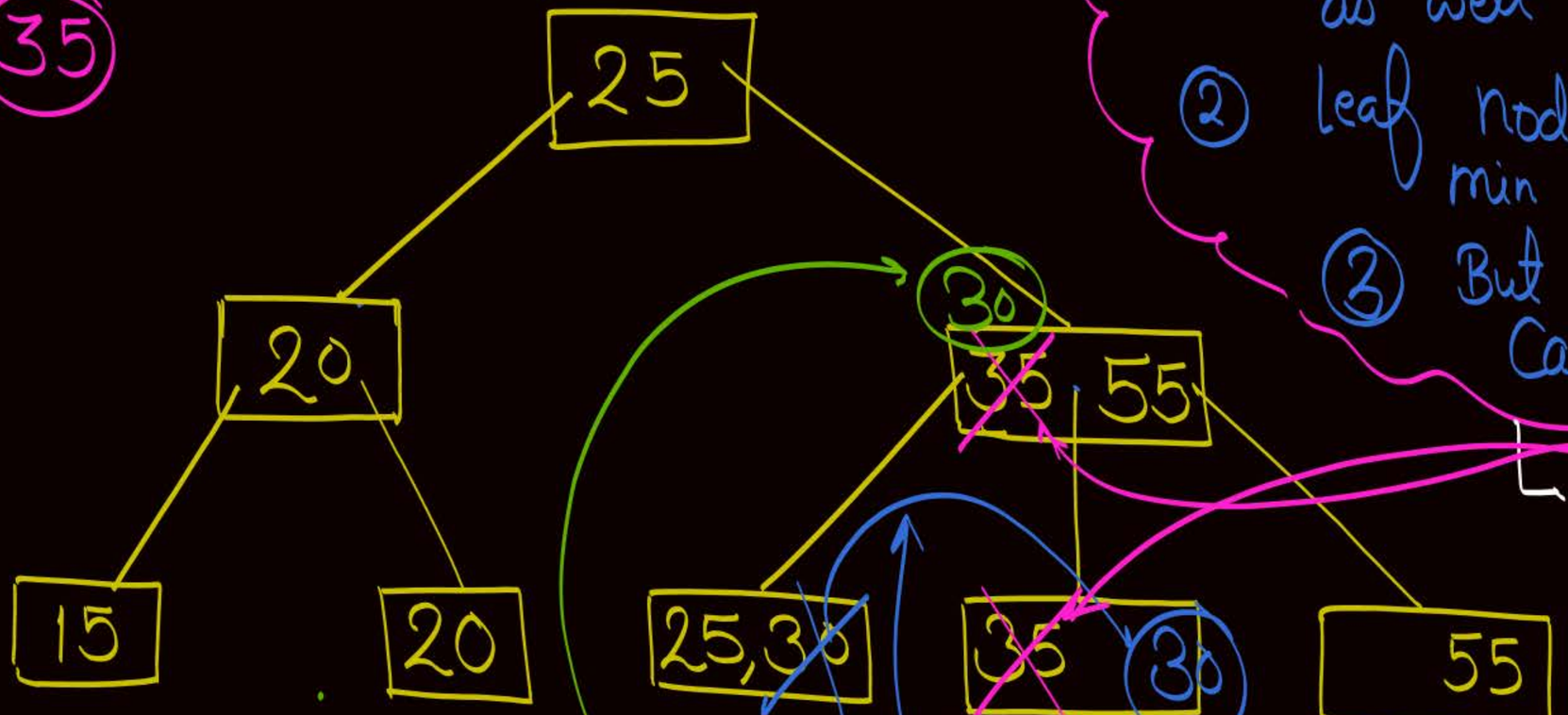
Delete: ~~40~~, ~~5~~, 45, 35, 25, 55

Final tree after deletion of (45)



Delete: ~~46~~, ~~5~~, ~~45~~, 35, 25, 55

delete (35)



① Key is present in leaf node as well as in internal node

② leaf node contains exact min no. of keys

③ But left sibling can help.

① Delete from both places

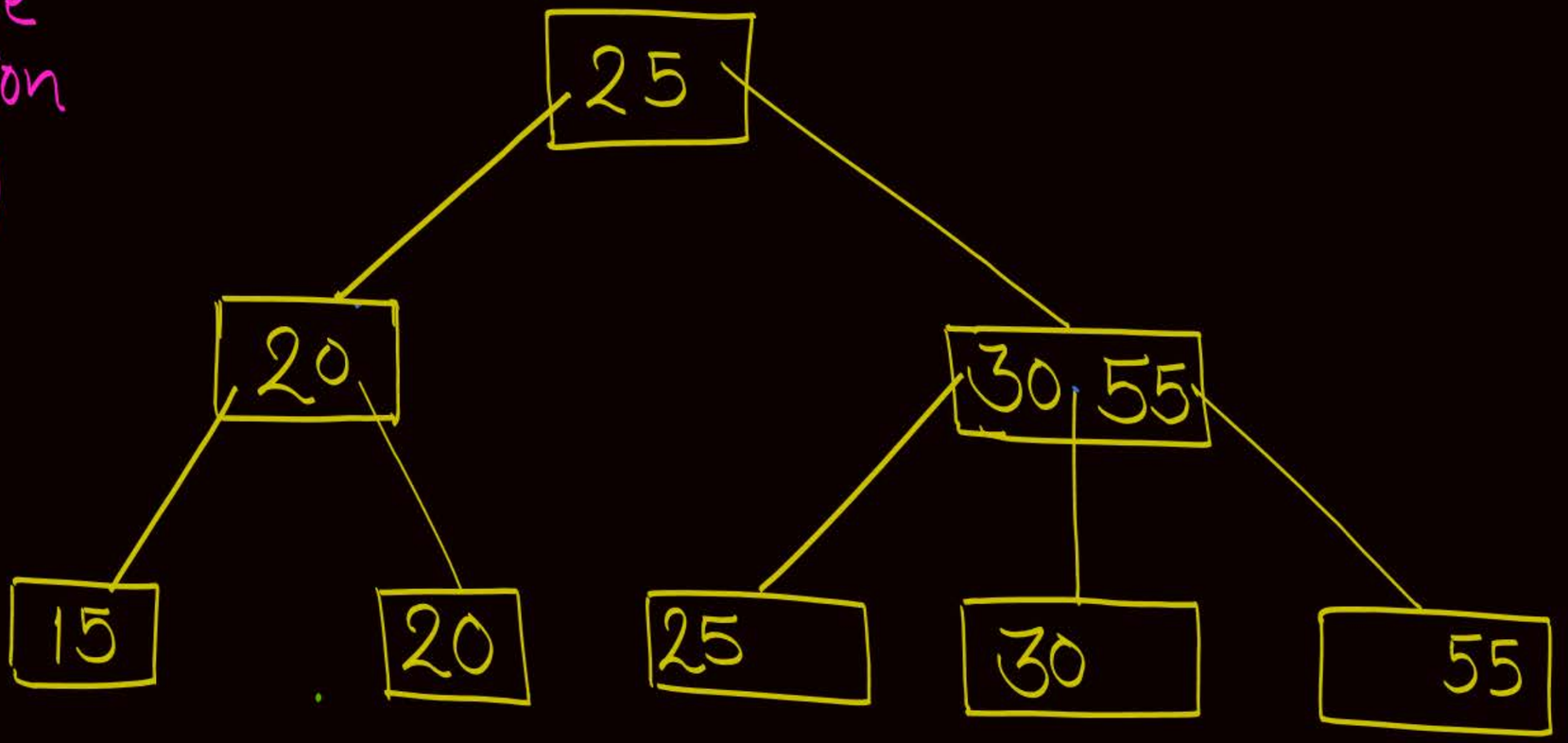
② Borrow from sibling

③ Update Key in parent if required

④ If Empty space is not handled then fill with successor (or Predecessor)

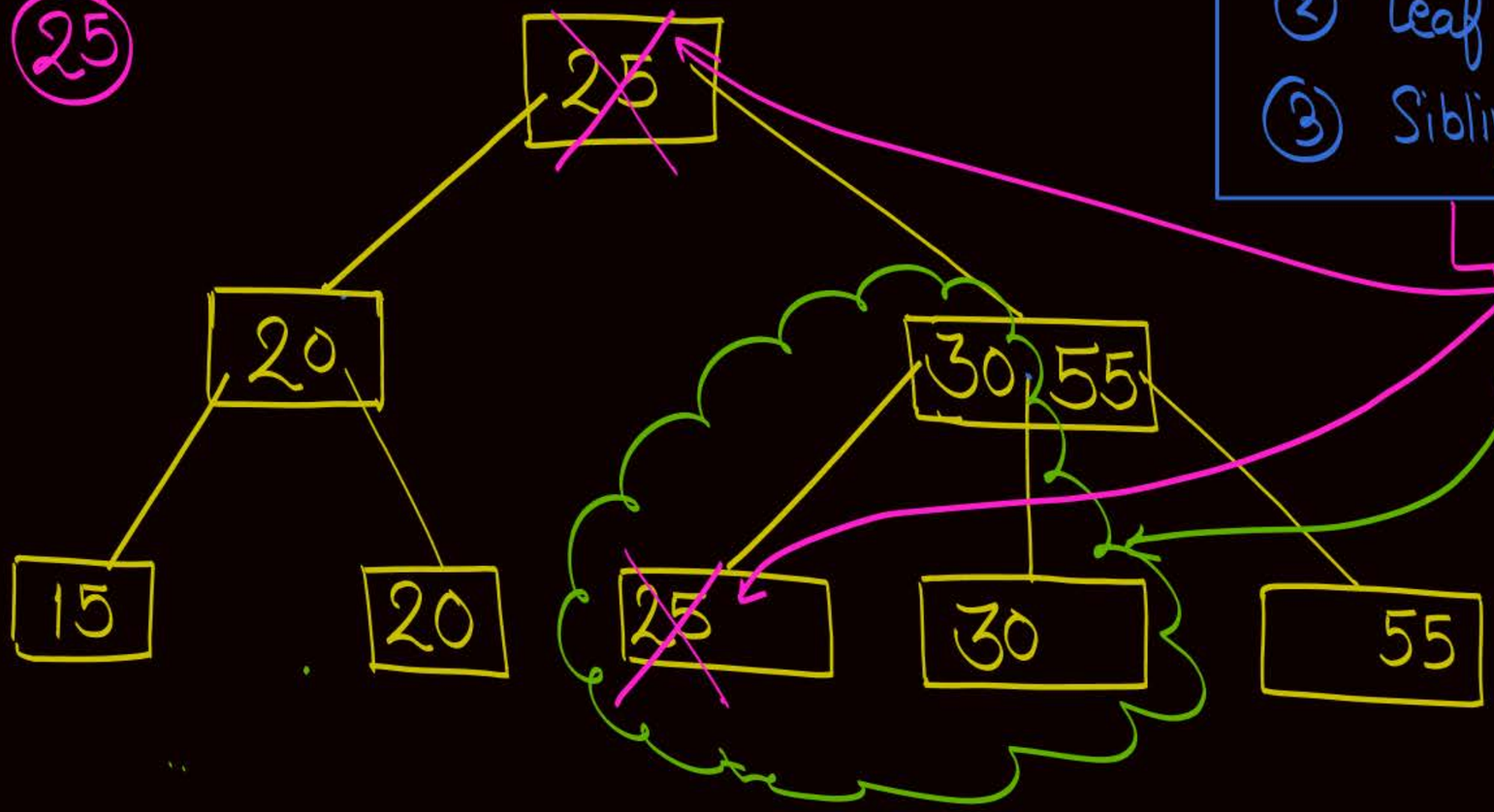
Delete: ~~40~~, ~~5~~, ~~45~~, 35, 25, 55

Final tree after deletion of (35)



Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, 25, 55

delete (25)

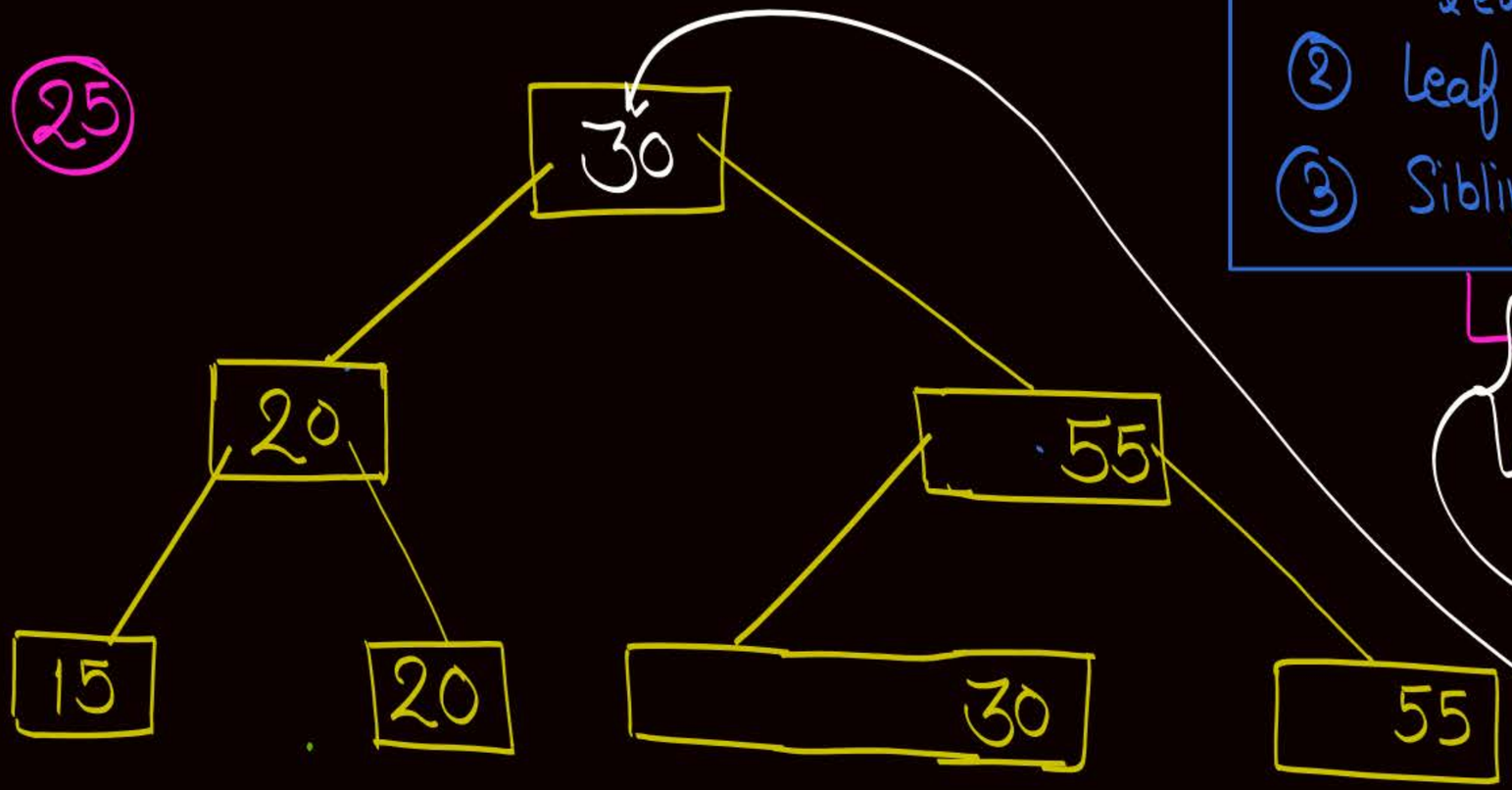


① '25' is present in leaf + internal
② Leaf Contains minimum required
③ Siblings can not help

└ (1) Delete from both places
(2) Merge with sibling

Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, 25, 55

delete (25)



① '25' is present in leaf + internal
② Leaf contains minimum required
③ Siblings can not help

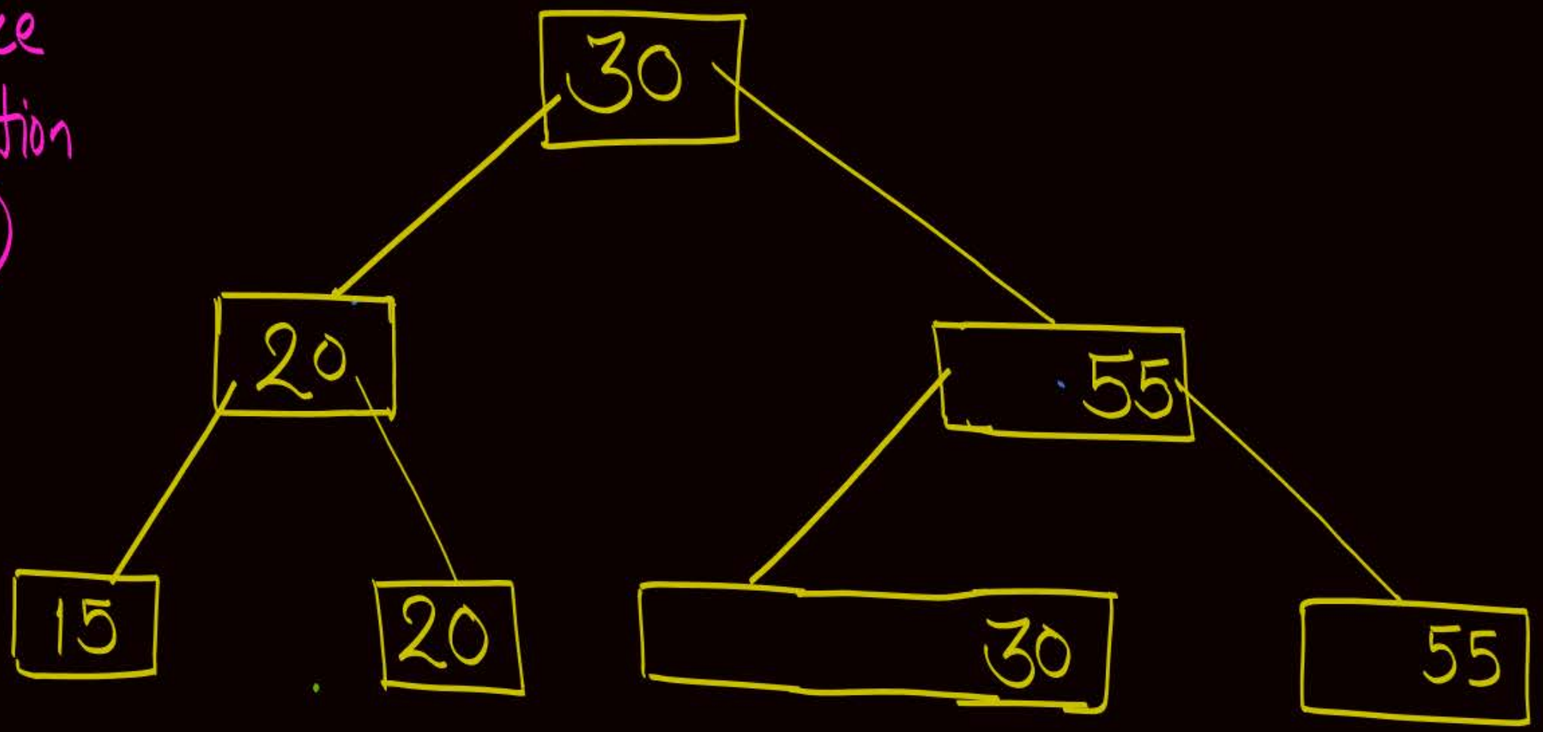
↳ (1) Delete from both places
(2) Merge with sibling

Empty space created in internal node is not yet handled

↳ ∴ Fill it with inorder successor
-('=' in RH subtree)

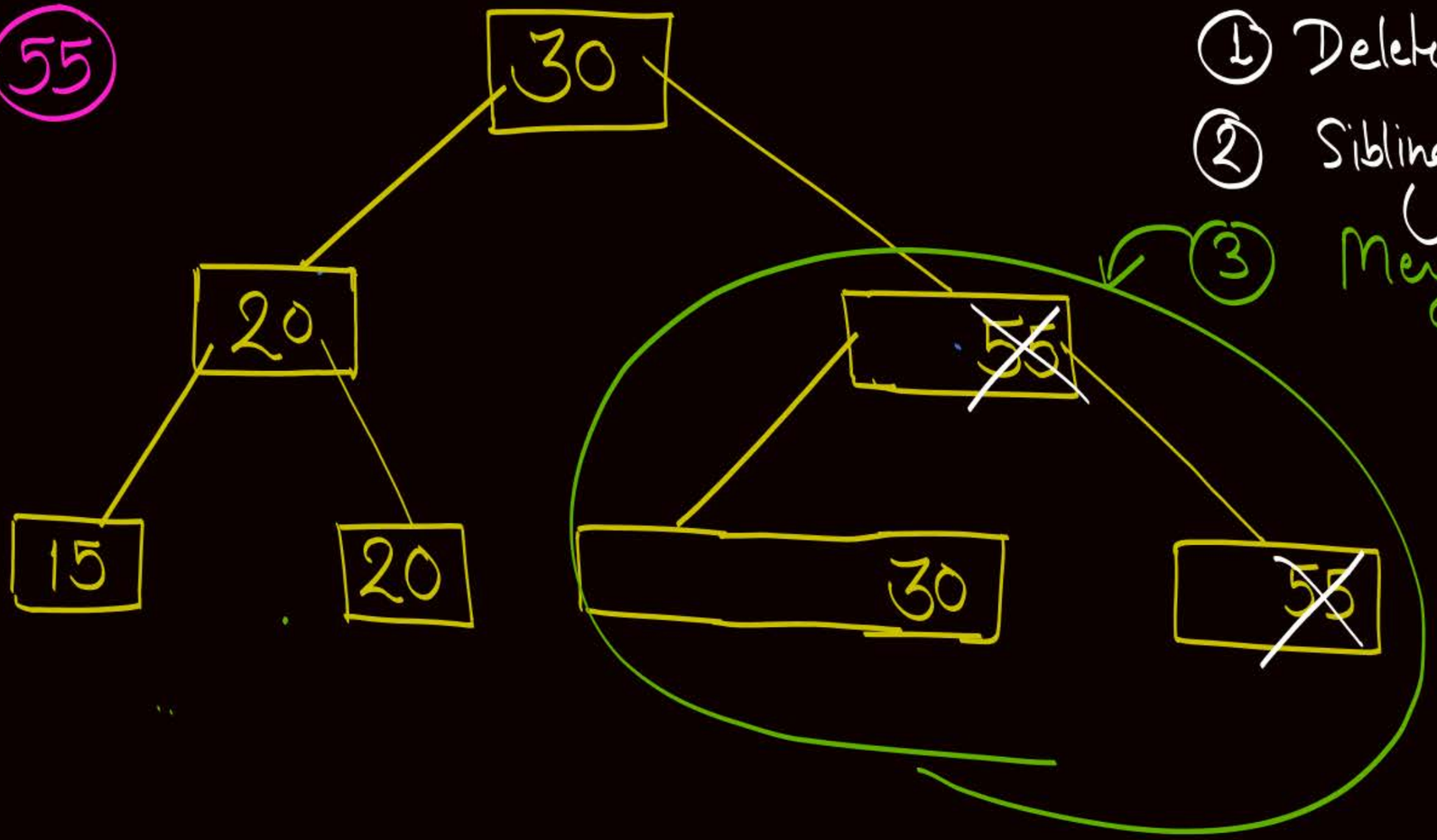
Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, 25, 55

Final tree after deletion of (25)



Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, ~~25~~, 55

delete (55)

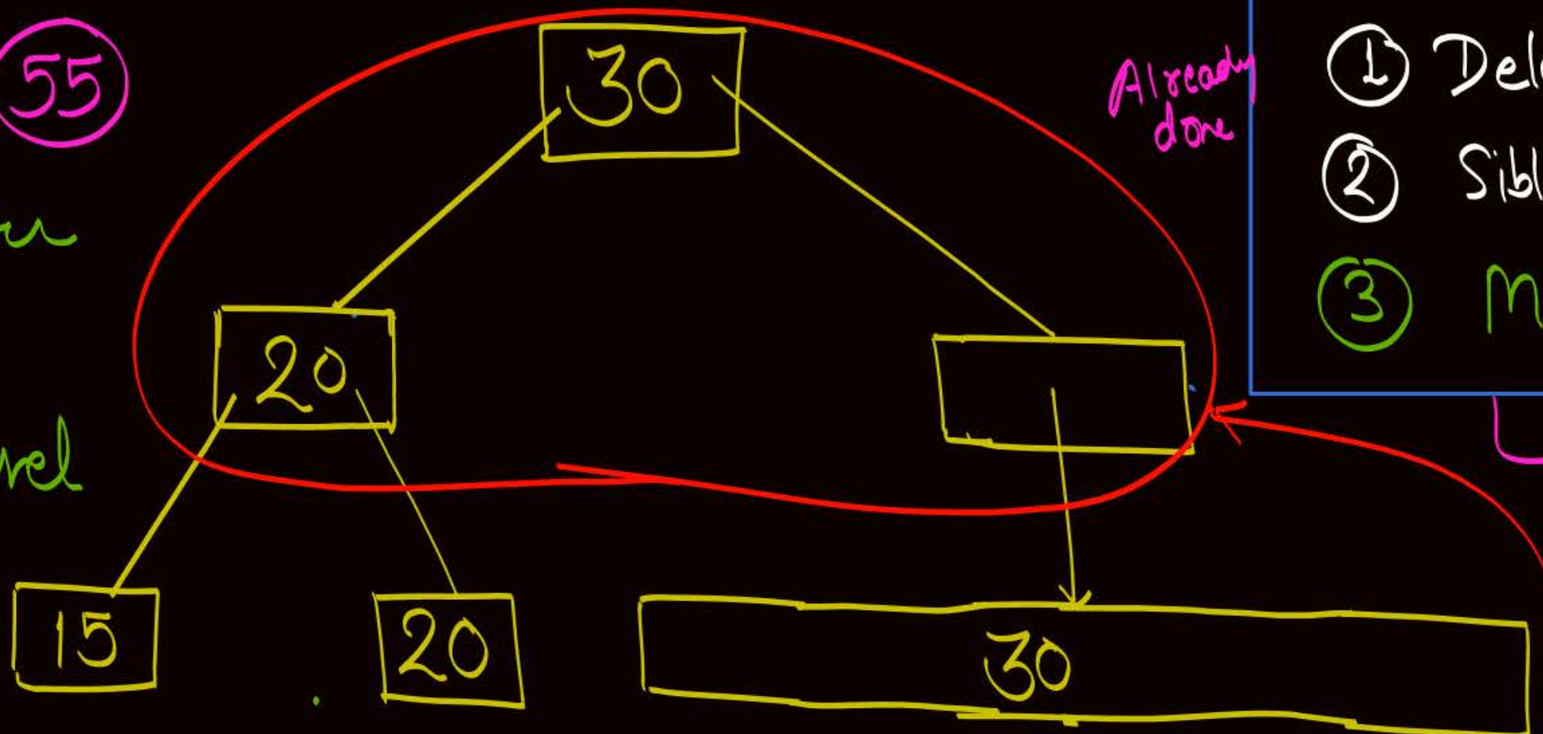


1. Delete from both places
2. Sibling can not help
3. Merge with sibling

Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, ~~25~~, 55

delete (55)

tree after merging at leaf level



Already done

① Delete from both places
② Sibling can not help
③ Merge with sibling

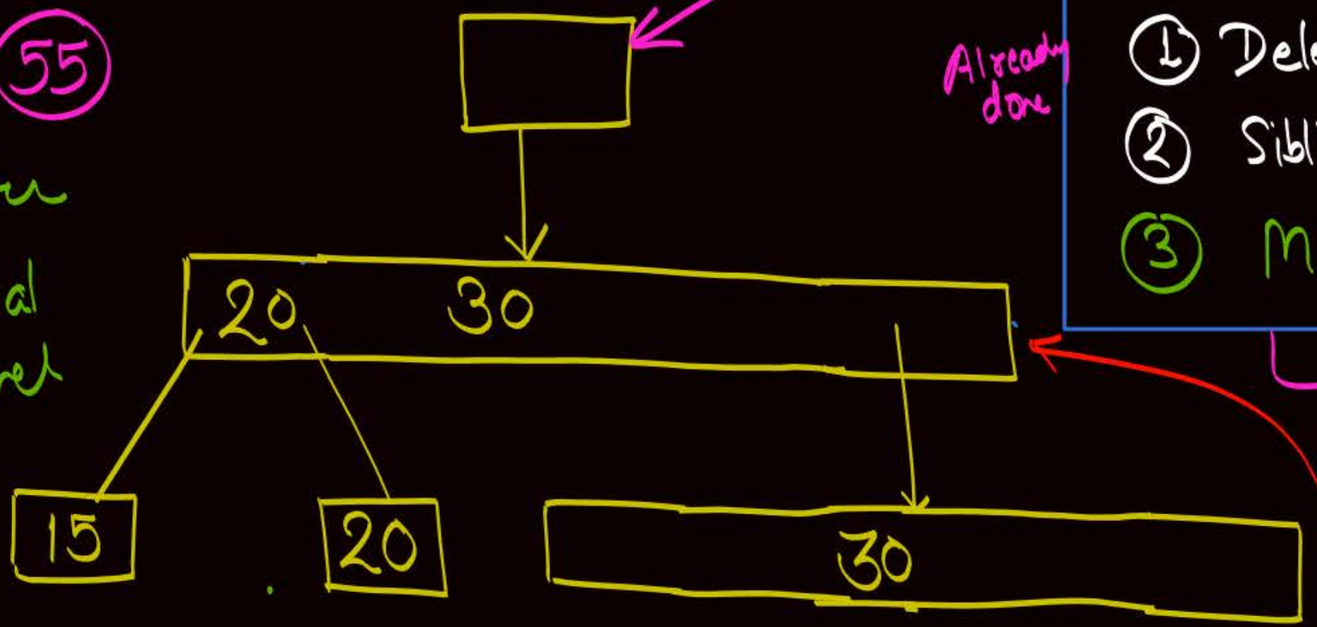
Now

ⓐ Parent node is deficient
ⓑ Sibling of parent node can not help.
ⓒ ∴ Merge with sibling

Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, ~~20~~, 55

delete (55)

tree after merging at Parent level



Already done

① Delete from both places
② Sibling can not help
③ Merge with sibling

Now

ⓐ Parent node is deficient
ⓑ Sibling of parent node can not help.
ⓒ ∴ Merge with sibling

⇒ Now root node becomes deficient

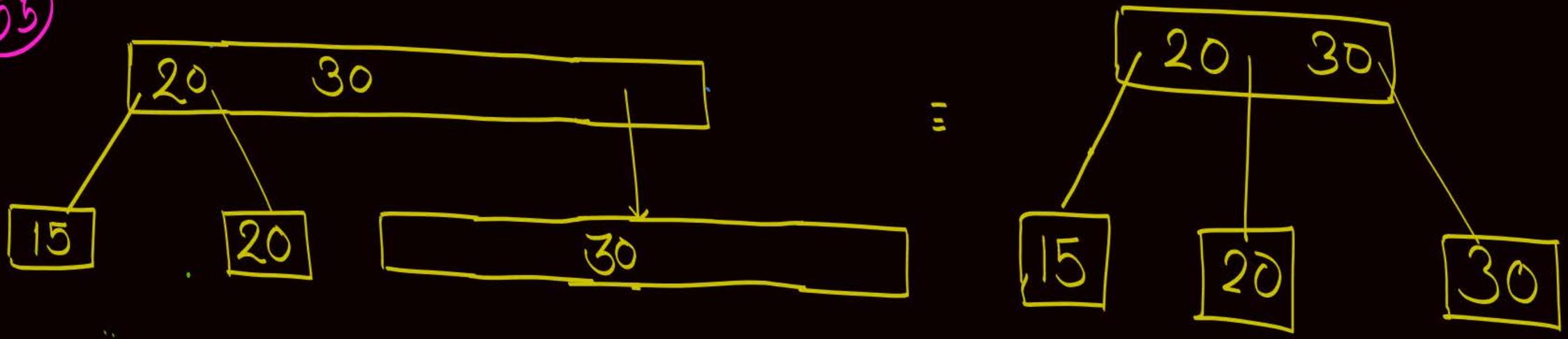
⇓

∴ delete current root

Height ↓ decrease

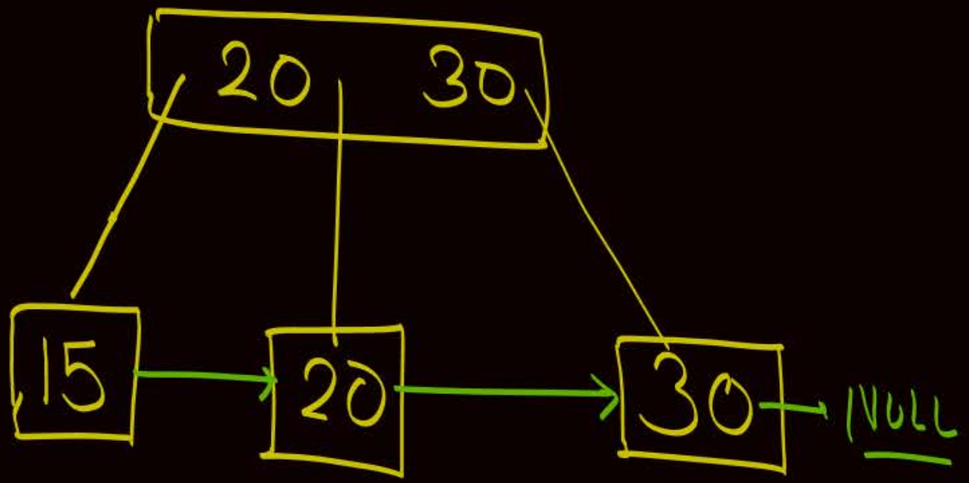
Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, ~~20~~, 55

Final tree after deletion of (55)



Delete: ~~40~~, ~~5~~, ~~45~~, ~~35~~, ~~25~~, ~~55~~

Final tree after all deletions





2 mins Summary

Topic Deletion from B+ tree

Slide

THANK - YOU